OKE Network Design Layout

Recommended OCI Console layout for an OKE cluster with a public Kubernetes API endpoint and private worker and pod networking. Region: ap-hyderabad-1.

1. Design summary

Item	Design decision
Region	ap-hyderabad-1
Public subnet use	Kubernetes API endpoint
Private subnet use	Worker node pool
Dedicated pod subnet	Required only for VCN-native pod networking
Internet Gateway	Used only for the public API endpoint subnet
NAT Gateway	Provides outbound access (including OCI services such as OCIR) for private worker nodes and pods
Pod-to-pod communication	Allow pod CIDR traffic with NSG or security rules
Flannel overlay note	With Flannel overlay, pods do not need a separate VCN pod subnet

2. Recommended CIDR layout

Resource	CIDR	Type
VCN	10.0.0.0/16	VCN
API endpoint subnet (oke-api-public)	10.0.0.0/29	Public
Worker node subnet (oke-workers-private)	10.0.1.0/24	Private
Pod subnet (oke-pods-private)	10.0.32.0/19	Private
Load balancer subnet (oke-lb-public) - optional, add only if you expose Services of type LoadBalancer	10.0.2.0/24	Public

3. Build order (dependency chain)

Create resources in this order. Route rules can only target gateways that already exist, and a subnet needs its route table at creation time.

Order	Resource	Depends on
1	VCN	-
2	Internet Gateway, NAT Gateway	VCN
3	Route tables (rt-public, rt-private)	Gateways
4	Subnets (API, workers, pods, optional LB)	VCN, route tables
5	NSGs and NSG rules	VCN, subnet CIDRs
6	OKE cluster	Subnets, nsg-api
7	Node pool	Cluster, worker subnet, NSGs

4. Target topology

Component	Purpose	Connects to
Internet	External access path	Internet Gateway
Internet Gateway	Public ingress and egress for the API endpoint subnet	Public API Endpoint Subnet (via rt-public)

Component	Purpose	Connects to
Public API Endpoint Subnet	Hosts the OKE Kubernetes API endpoint	OKE Kubernetes API
Private Worker Subnet	Hosts worker nodes with private IP addresses only	NAT Gateway (via rt-private)
Private Pod Subnet	Hosts pod IPs for VCN-native pod networking	NAT Gateway (via rt-private)
Pod NSG or security rules	Allow pod-to-pod and pod-to-worker traffic	Pods and worker nodes

5. OCI Console steps

Step 1: Create the VCN

- Open the OCI Console and select region ap-hyderabad-1.
- Go to Networking > Virtual Cloud Networks and select the correct compartment.
- Click Create VCN (manual creation is recommended, so the order of resources is under your control).
- Enter a VCN name, set CIDR to 10.0.0.0/16, and enable DNS resolution. Create the VCN.

If you use Start VCN Wizard instead, it creates its own gateways, route tables and subnets. Review them against the names and CIDRs in this document to avoid duplicates.

Step 2: Create the gateways

- From the VCN details page, open Resources.
- Internet Gateway: create **oke-igw** and enable it. It is used for the public API endpoint subnet only.
- NAT Gateway: create **oke-nat** for outbound internet access from private worker nodes and pods.

Gateways must exist before any route table can reference them.

Step 3: Create the route tables

- Open VCN > Resources > Route Tables > Create Route Table.
- **rt-public** - one rule: 0.0.0.0/0 -> Internet Gateway (oke-igw). Used by the API endpoint subnet and the optional LB subnet.
- **rt-private** - one rule: 0.0.0.0/0 -> NAT Gateway (oke-nat). Used by the worker subnet and the pod subnet.

Route tables must exist before the subnets, because each subnet is attached to its route table at creation time.

Step 4: Create the public API endpoint subnet

- Create subnet **oke-api-public** as a Regional subnet with CIDR 10.0.0.0/29.
- Subnet access: Public. Route table: **rt-public** (0.0.0.0/0 -> oke-igw).
- Allow Kubernetes API access only from trusted administrator, VPN, bastion, or CI/CD source CIDRs. Avoid unrestricted access to TCP 6443.

Step 5: Create the private worker node subnet

- Create subnet **oke-workers-private** as a Regional subnet with CIDR 10.0.1.0/24; do not assign public IPv4 addresses (private subnet).
- Route table: **rt-private** (0.0.0.0/0 -> oke-nat).

Step 6: Create the private pod subnet (VCN-native pod networking only)

- Create **oke-pods-private** only if you will use VCN-native pod networking, with CIDR 10.0.32.0/19, private.
- Route table: **rt-private** (same outbound route: 0.0.0.0/0 -> NAT Gateway).
- Make sure the pod subnet does not overlap the other subnets in the VCN, the Kubernetes service CIDR, or any connected on-premises and VPN networks.

Step 7: Create the load balancer subnet (optional)

- Create **oke-lb-public** (10.0.2.0/24, Regional, public) only if you expose Services of type LoadBalancer.
- Route table: **rt-public** (0.0.0.0/0 -> oke-igw).

Step 8: Configure security rules

- Prefer NSGs over security lists.
- Create four NSGs: nsg-api, nsg-workers, nsg-pods and (optional) nsg-lb. The full rule sets are in Section 6.
- Security rules are stateful by default, so return traffic for established connections is allowed automatically. If you use stateless rules, add the matching return rules.

Step 9: Create the OKE cluster

- Go to Developer Services > Kubernetes Clusters (OKE) > Create Cluster > Custom Create.
- Select your VCN and choose oke-api-public for the Kubernetes API endpoint. Enable a public IP for the endpoint if the Console offers it; attach nsg-api.
- Choose the networking type: VCN-native pod networking (dedicated pod subnet) or Flannel overlay (no dedicated pod subnet).
- Choose oke-workers-private for worker nodes, and oke-pods-private for the pod subnet when using VCN-native pod networking.
- Create the cluster.

Step 10: Create the node pool

- Open the cluster > Node pools > Create node pool.
- Select subnet oke-workers-private and use private IP addresses only.
- Select the shape, image, Kubernetes version, and node count. Attach nsg-workers (and nsg-pods for VCN-native).
- Create the node pool.

Important: the Internet Gateway is for the public API endpoint subnet (and optional LB subnet) only. Private worker and pod subnets use the NAT Gateway for all outbound access, including OCI services.

6. Security rules (NSG), including ports 443, 22 and all OKE-required ports

All rules below are NSG rules. Replace ADMIN_CIDR with your office, VPN, bastion or CI/CD public CIDR. Do not use 0.0.0.0/0 for TCP 6443 or TCP 22.

6.1 API endpoint NSG (nsg-api)

Ingress

Source	Protocol	Dest. port	Purpose
10.0.1.0/24 (workers)	TCP	6443	Workers to Kubernetes API
10.0.1.0/24 (workers)	TCP	12250	Workers to OKE control plane (kubelet/proxymux)
10.0.32.0/19 (pods)	TCP	6443, 12250	Pods to Kubernetes API (VCN-native)
ADMIN_CIDR	TCP	6443	kubectl, CI/CD, VPN, bastion access
10.0.1.0/24 (workers)	ICMP	Type 3, Code 4	Path MTU discovery

Egress

Destination	Protocol	Dest. port	Purpose
10.0.1.0/24 (workers)	TCP	10250	API to kubelet (logs, exec, metrics)
10.0.32.0/19 (pods)	TCP	All	API to pods (webhooks, VCN-native)
10.0.1.0/24 (workers)	ICMP	Type 3, Code 4	Path MTU discovery

6.2 Worker node NSG (nsg-workers)

Ingress

Source	Protocol	Dest. port	Purpose
10.0.1.0/24 (workers)	All	All	Node-to-node (includes Flannel VXLAN UDP 8472)
10.0.32.0/19 (pods)	All	All	Pod-to-worker traffic
10.0.0.0/29 (API endpoint)	TCP	10250	Kubelet API from control plane
10.0.0.0/29 (API endpoint)	TCP	6443, 12250	Control plane to workers where required
ADMIN_CIDR / bastion CIDR	TCP	22	SSH to worker nodes (restricted)
10.0.2.0/24 (LB subnet, optional)	TCP	30000-32767	NodePort range from load balancer
10.0.2.0/24 (LB subnet, optional)	TCP	10256	kube-proxy health check from load balancer
0.0.0.0/0	ICMP	Type 3, Code 4	Path MTU discovery

Egress

Destination	Protocol	Dest. port	Purpose
10.0.1.0/24 (workers)	All	All	Node-to-node
10.0.32.0/19 (pods)	All	All	Worker-to-pod traffic
10.0.0.0/29 (API endpoint)	TCP	6443, 12250	Kubelet/kube-proxy to API and control plane
10.0.0.0/29 (API endpoint)	ICMP	Type 3, Code 4	Path MTU discovery
0.0.0.0/0 (NAT Gateway)	TCP	443, 80	HTTPS/HTTP to the internet and OCI services (OCIR, public registries, updates)
0.0.0.0/0 (NAT Gateway)	UDP / TCP	53, 123	DNS and NTP, if you restrict egress further

6.3 Pod NSG (nsg-pods, VCN-native pod networking only)

Ingress

Source	Protocol	Dest. port	Purpose
10.0.32.0/19 (pods)	All	All	Pod-to-pod
10.0.1.0/24 (workers)	All	All	Worker-to-pod
10.0.0.0/29 (API endpoint)	TCP	All	API server to pods (webhooks)
10.0.2.0/24 (LB subnet, optional)	TCP	Pod service ports	Load balancer to pods (VCN-native backends)
0.0.0.0/0	ICMP	Type 3, Code 4	Path MTU discovery

Egress

Destination	Protocol	Dest. port	Purpose
10.0.32.0/19 (pods)	All	All	Pod-to-pod
10.0.1.0/24 (workers)	All	All	Pod-to-worker
10.0.0.0/29 (API endpoint)	TCP	6443, 12250	Pods to Kubernetes API and control plane
0.0.0.0/0 (NAT Gateway)	TCP	443, 80	Outbound internet and OCI services

6.4 Load balancer NSG (nsg-lb, optional)

Direction	Source / Dest.	Protocol	Port	Purpose
Ingress	0.0.0.0/0 (or restricted)	TCP	443, 80	Public HTTPS/HTTP to the application
Egress	10.0.1.0/24 (workers)	TCP	30000-32767, 10256	NodePort traffic and health checks
Egress	10.0.32.0/19 (pods)	TCP	Pod service ports	Direct to pods (VCN-native)

6.5 Quick port reference

Port	Protocol	Used for
22	TCP	SSH to worker nodes (admin/bastion CIDR only)
443	TCP	HTTPS: OCI services, image registries, public ingress
6443	TCP	Kubernetes API server
12250	TCP	OKE control plane communication (kubelet / proxymux)
10250	TCP	Kubelet API
10256	TCP	kube-proxy health check (load balancers)
30000-32767	TCP	Kubernetes NodePort range
8472	UDP	Flannel VXLAN overlay between workers
ICMP 3,4	ICMP	Path MTU discovery

Notes: with Flannel overlay, skip the pod subnet and nsg-pods; pod traffic is carried between workers (covered by the worker-to-worker all-protocols rule). The pod subnet 10.0.32.0/19 is carved from inside the VCN CIDR, so "no overlap" means no overlap with other subnets, the service CIDR, and connected networks. Verify the final rules against the Oracle documentation below before production use.

7. Sources

https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengnetworkconfig.htm
https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengnetworkconfigexample.htm
https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengcreatingclusterusingoke_topic-Using_the_Console_to_create_a_Custom_Cluster_with_Explicitly_Defined_Settings.htm
https://docs.oracle.com/en-us/iaas/Content/Network/Tasks/quickstartnetworking.htm